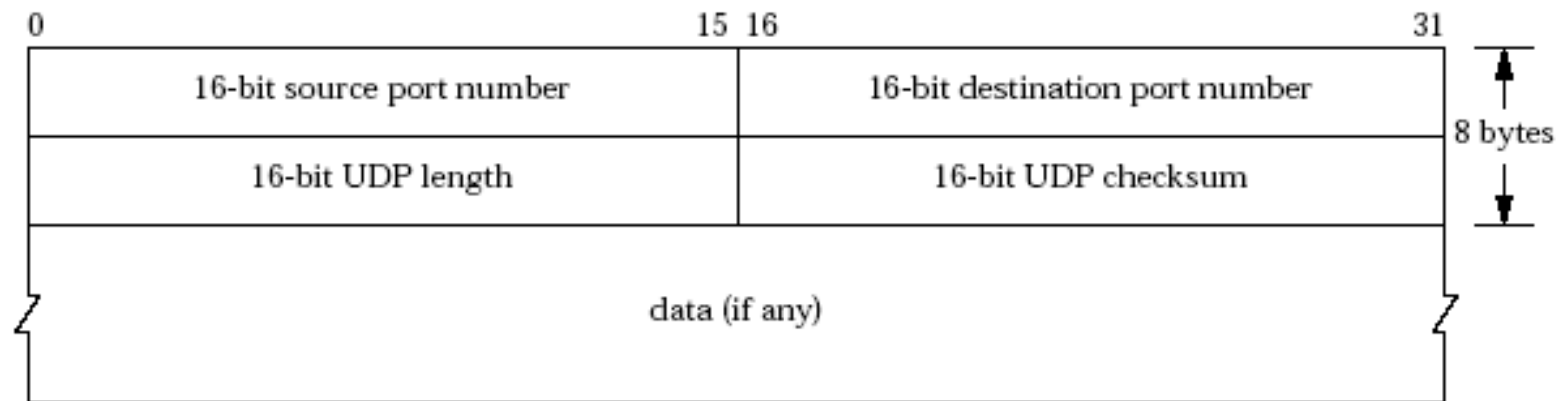


Bài 07:

UDP Socket

UDP

- ❑ Dịch vụ không kết nối
- ❑ Không cung cấp: thiết lập kết nối, tin cậy, điều khiển dòng, thời gian, ...
- ❑ Tại sao sử dụng UDP?



UDP

❑ Nhược điểm

- Các thông điệp có thể được nhận theo bất kỳ thứ tự nào.
- Không có gì đảm bảo là các gói tin sẽ đến đích

❑ Ưu điểm:

- UDP một giao thức có tốc độ truyền tin nhanh
- Truyền tin unicast, broadcast và multicast.

❑ Kiểu truyền

- Một thông điệp unicast được gửi từ nút này tới nút khác.
- Truyền tin broadcast nghĩa là một thông điệp có thể được gửi tới tất cả các nút trong một mạng.
- Multicast cho phép các thông điệp được truyền tới một nhóm các nút được lựa chọn.

Lập trình với Socket UDP

UDP: không kết nối giữa client và server

- ❑ Không bắt tay
- ❑ Phía gửi sẽ gắn địa chỉ IP và Port của đích vào mỗi đoạn dữ liệu.
- ❑ OS sẽ đính kèm IP và Port của máy gửi vào mỗi đoạn.
- ❑ Server loại bỏ phần IP và port của máy gửi và nhận đoạn dữ liệu.

Góc độ ứng dụng

UDP cung cấp cách thức gửi không đảm bảo một nhóm byte ("datagrams") giữa client và server

Ví dụ

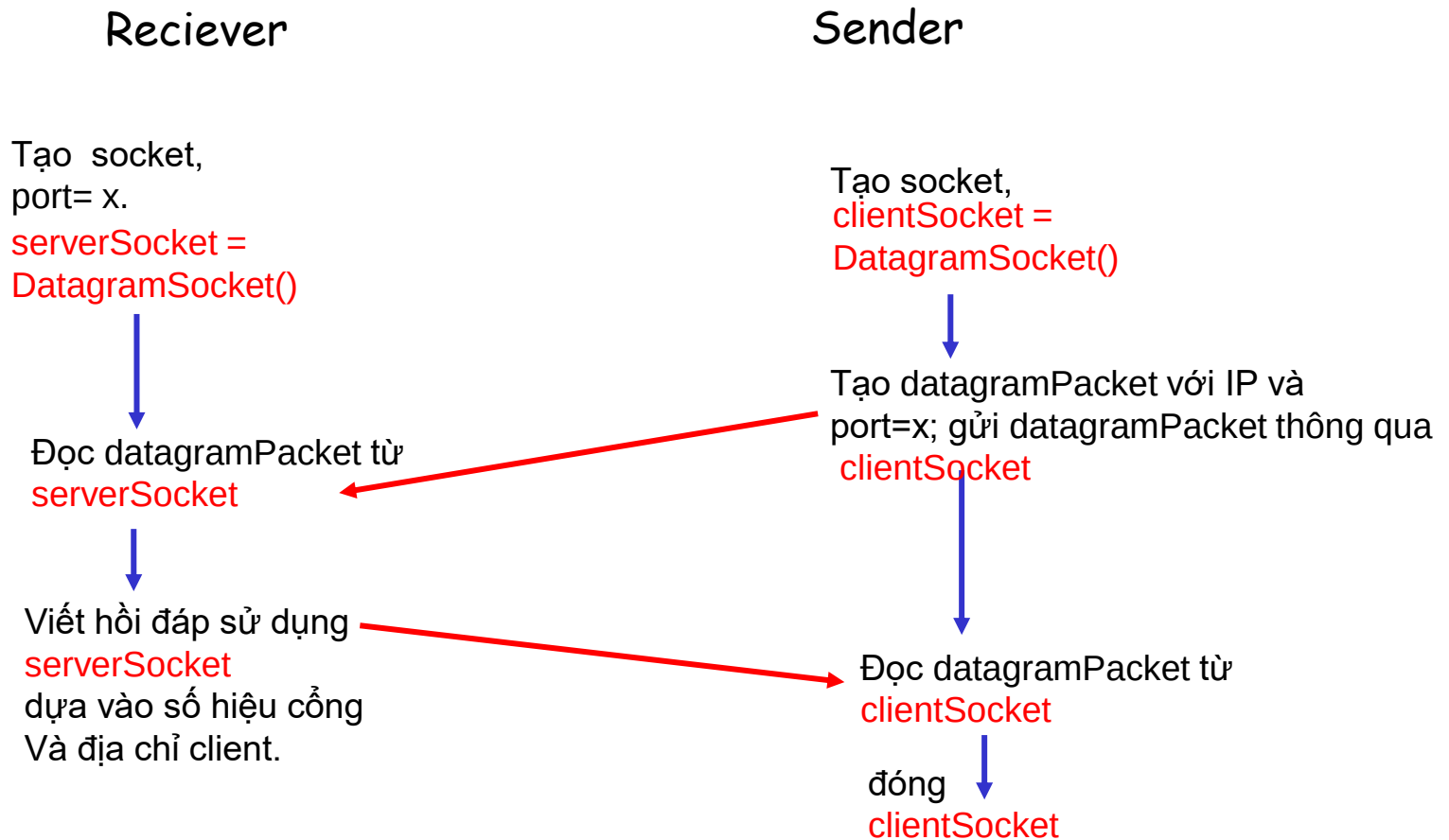
□ Client (Sender):

- Người dùng gõ một đoạn văn bản.
- Client gửi dòng văn bản tới server.
- Nhận dòng văn bản đã sửa đổi.
- Hiển thị.

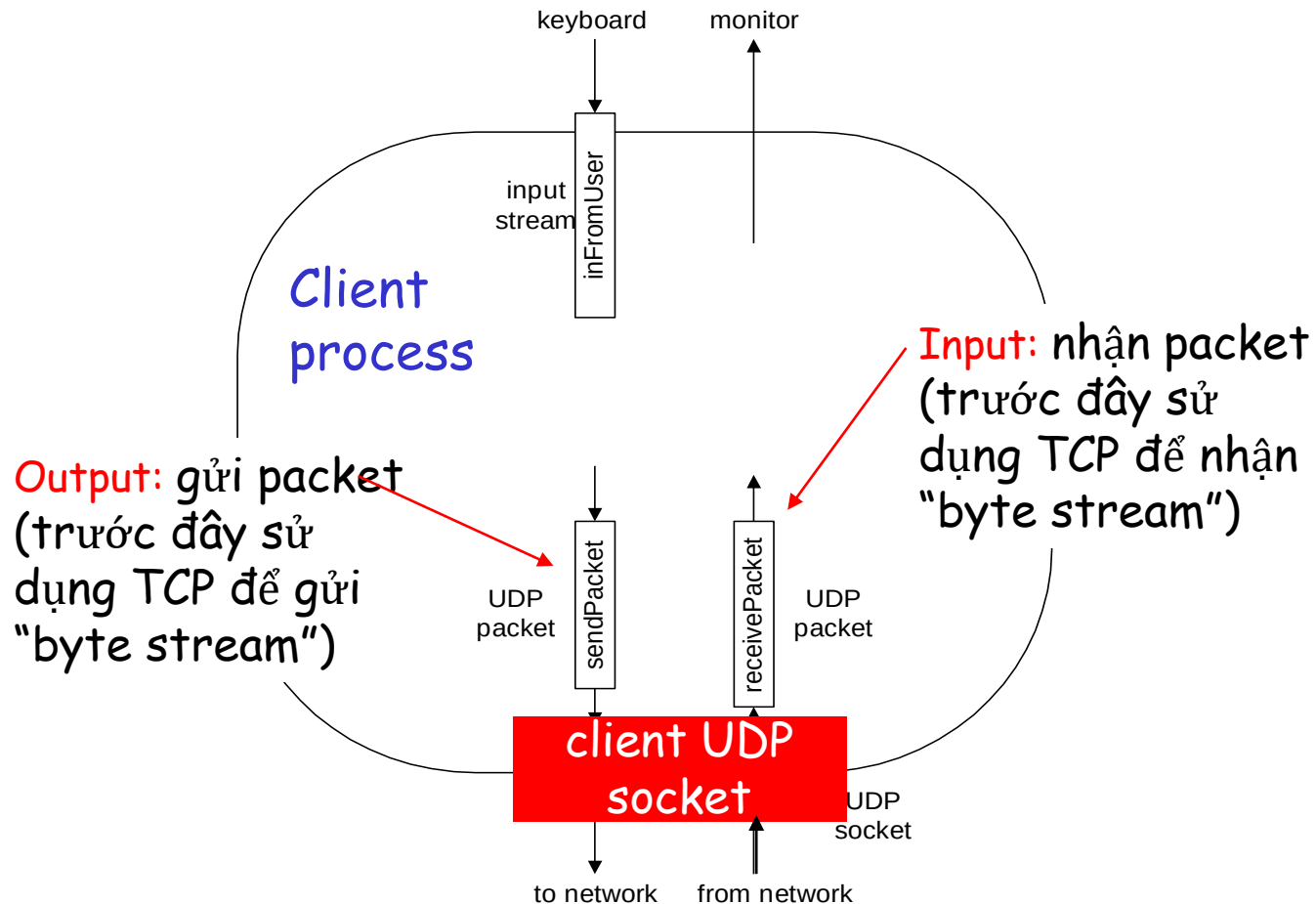
□ Server (Receiver):

- Server nhận dòng văn bản từ client.
- Đổi hoa tất cả các ký tự.
- Gửi đoạn văn bản đã sửa đổi về cho client.

Tương tác giữa socket sender/recvier



Ví dụ: Java client (UDP)



Ví dụ: Java client (UDP)

```
import java.io.*;  
import java.net.*;
```

```
class UDPClient {  
    public static void main(String args[]) throws Exception  
    {
```

Tạo
Luồng nhập

```
        BufferedReader inFromUser =  
            new BufferedReader(new InputStreamReader(System.in));
```

Tạo socket client

Dịch tên miền

hostname sang
địa chỉ IP
sử dụng DNS

```
        DatagramSocket clientSocket = new DatagramSocket();
```

```
        InetAddress IPAddress = InetAddress.getByName("localhost");
```

```
        byte[] sendData = new byte[1024];  
        byte[] receiveData = new byte[1024];
```

```
        String sentence = inFromUser.readLine();  
        sendData = sentence.getBytes();
```

Ví dụ: Java client UDP

Tạo datagram với
dữ liệu, độ dài dữ
liệu, địa chỉ IP,
số hiệu cổng

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress, 9876);
```

Gửi datagram
Tới server

```
clientSocket.send(sendPacket);
```

```
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);
```

Đọc datagram
từ server

```
clientSocket.receive(receivePacket);
```

```
String modifiedSentence =  
    new String(receivePacket.getData());
```

```
System.out.println("FROM SERVER:" + modifiedSentence);  
clientSocket.close();  
}
```

```
}
```

Ví dụ: Java server UDP

```
import java.io.*;  
import java.net.*;
```

```
class UDPServer {  
    public static void main(String args[]) throws Exception  
    {
```

Tạo datagram socket
Lắng nghe tại
port 9876



```
DatagramSocket serverSocket = new DatagramSocket(9876);
```

```
byte[] receiveData = new byte[1024];  
byte[] sendData = new byte[1024];
```

```
while(true)  
{
```

Cấp phát bộ nhớ
để nhận gói dữ liệu



```
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);
```

Nhận gói
dữ liệu



```
serverSocket.receive(receivePacket);
```

Ví dụ: Java server UDP

```
String sentence = new String(receivePacket.getData());
```

Lấy địa chỉ IP,
port của gói gửi

```
InetAddress IPAddress = receivePacket.getAddress();  
int port = receivePacket.getPort();
```

```
String capitalizedSentence = sentence.toUpperCase();
```

```
sendData = capitalizedSentence.getBytes();
```

Tạo datagram
để gửi tới client

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress,  
        port);
```

Viết
Datagram
ra socket

```
serverSocket.send(sendPacket);  
}  
}
```

Kết thúc lặp
và đợi gói dữ liệu khác

Quan sát và trả lời các câu hỏi về UDP

- ❑ DatagramSocket được sử dụng bởi Server hay Client?
- ❑ Client gửi đoạn dữ liệu tới server mà không cần địa chỉ IP và port?
- ❑ Địa chỉ IP và port của máy đích được gửi kèm trong mỗi đoạn dữ liệu?
- ❑ Nhiều client có thể dùng một Server?

DatagramPacket

- ❑ Kích thước tối đa một gói dữ liệu là 65,508 byte
- ❑ Phương thức khởi tạo:

❖ **public DatagramPacket (byte[] buffer, int length)**

❖ **public DatagramPacket (byte[] buffer, int offset, int length)**

❖ **public DatagramPacket**

(byte[] data, int length, InetAddress destination, int port)

❖

DatagramPacket

□ Các phương thức khởi tạo khác:

❖ `public DatagramPacket`

`(byte[] data, int offset, int length, InetAddress destination, int port)`

❖ `public DatagramPacket`

`(byte[] data, int length, SocketAddress destination, int port)`

❖ `public DatagramPacket`

`(byte[] data, int offset, int length, SocketAddress destination, int port)`

Ví dụ

```
14 String s = "On essaie...";
15 byte[] data = s.getBytes("ASCII");
16
17 try {
18     InetAddress ia = InetAddress.getByName("it.hutech.edu.vn");
19     int port = 7;
20     DatagramPacket dp = new DatagramPacket(data, data.length, ia, port);
21 } catch (UnknownHostException ex) {
22 }
```

Các phương thức

❑ Thao tác với địa chỉ

- **public *InetAddress*** **getAddress()**
- **public *int*** **getPort()**
- **public *SocketAddress*** **getSocketAddress()**
- **public *void*** **setAddress(InetAddress remote)**
- **public *void*** **setPort(int port)**
- **public *void*** **setAddress(SocketAddress remote)**

Các phương thức

❑ Thao tác với dữ liệu

- **public** *byte[]* **getData ()**
- **public** *int* **getLength ()**
- **public** *int* **getOffset ()**
- **public** *void* **setData (byte[] data)**
- **public** *void* **setData (byte[] data, int offset, int length)**
- **public** *void* **setLength (int length)**

Ví dụ

```
10 public class DatagramExample {
11
12     public static void main(String[] args) {
13         String s = "Hello";
14         byte[] data = s.getBytes();
15         try {
16             InetAddress ia = InetAddress.getByName("www.hutech.edu.vn");
17             int port = 7;
18             DatagramPacket dp;
19             dp = new DatagramPacket(data, data.length, ia, port);
20             System.out.println("Address: " + dp.getAddress());
21             System.out.println("Port: " + dp.getPort());
22             System.out.println("Length: " + dp.getLength());
23             System.out.println("Data: ");
24             s = new String(dp.getData(), dp.getOffset(), dp.getLength());
25             System.out.println(s);
26         } catch (UnknownHostException e) {
27             System.err.println(e);
28         }
29     }
30 }
```

DatagramSocket

❑ Phương thức khởi tạo

- ❖ **public DatagramSocket () throws SocketException**
- ❖ **public DatagramSocket (int port) throws SocketException**
- ❖ **public DatagramSocket(int port, InetAddress interface) throws SocketException**
- ❖ **public DatagramSocket(SocketAddress interface) throws SocketException**
- ❖ **protected DatagramSocket(DatagramSocketImpl impl) throws SocketException**

Ví dụ

```
14 public class UDPPortScanner {
15     public static void main(String[] args) {
16         for (int port = 0; port <= 65535; port++) {
17             try {
18                 // kiểm tra có port nào không được dùng ko?
19                 //nếu có, ngoại lệ sẽ được ném ra
20                 DatagramSocket server = new DatagramSocket(port);
21                 server.close( );
22             }
23             catch (SocketException ex) {
24                 System.out.println("Port:" + port + " đã được dùng");
25             } // end try
26         } // end for
27     }
28 }
29 }
```

Gửi và nhận

- ❖ **public void send (DatagramPacket dp) throws IOException**
- ❖ **public void receive (DatagramPacket dp) throws IOException**

Các phương thức khác

- ❑ public *void* close()
- ❑ public *int* getLocalPort()
- ❑ public *InetAddress* getLocalAddress()
- ❑ public *SocketAddress* getLocalSocketAddress()
- ❑ public *void* connect(InetAddress host, int port)
- ❑ public *void* disconnect()
- ❑ public *void* disconnect()
- ❑ public *int* getPort()
- ❑ public *InetAddress* getInetAddress()
- ❑ public *InetAddress* getRemoteSocketAddress()

UDPClientChatEx1

```
class UDPClientChatEx1 {  
12  
13     public static void main(String[] args) throws Exception {  
14         BufferedReader in = new BufferedReader(new InputStreamReader(System.in));  
15         InetAddress ipServer = InetAddress.getByName("localhost"); //serverName  
16         byte buffer[] = new byte[1024];  
17         int portServer = 8888;  
18         try {  
19             DatagramSocket socket = new DatagramSocket();  
20             DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
21             boolean ketthuc = false;  
22             String stgui = "", stnhan = "";  
23             do { //Gui  
24                 byte sendData[];  
25                 System.out.print("Client: ");  
26                 stgui = in.readLine();  
27                 sendData = stgui.getBytes();  
28                 DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, ipServer, portServer);  
29                 socket.send(sendPacket);  
30                 if (!stgui.equalsIgnoreCase("bye")) { //Nhan  
31                     socket.receive(packet);  
32                     stnhan = new String(packet.getData(), 0, packet.getLength());  
33                     System.out.println("Server: " + stnhan);  
34                 }  
35                 if (stgui.equalsIgnoreCase("bye") || stnhan.equalsIgnoreCase("bye")) {  
36                     ketthuc = true;  
37                 }  
38             } while (!ketthuc); //ketthuc=false  
39             socket.close();  
40         } catch (UnknownHostException e) {  
41             ;  
42         }  
43     }  
}
```

UDPServerChatEx1

```
class UDPServerChatEx1 {  
12  
13     public static void main(String[] args) throws Exception {  
14         BufferedReader in = new BufferedReader(new InputStreamReader(System.in));  
15         byte buffer[] = new byte[1024];  
16         int port = 8888;  
17         try {  
18             DatagramSocket socket = new DatagramSocket(port);  
19             System.out.println("May chu doi ket noi o cong 8888...");  
20             DatagramPacket packet = new DatagramPacket(buffer, buffer.length);  
21             boolean ketthuc = false;  
22             String stgui = "", stnhan = "";  
23             do { //Nhan  
24                 socket.receive(packet);  
25                 stnhan = new String(packet.getData(), 0, packet.getLength());  
26                 System.out.println("Client: " + stnhan);  
27                 if (!stnhan.equalsIgnoreCase("bye")) { //Gui  
28                     InetAddress ipClient = packet.getAddress();  
29                     int portClient = packet.getPort();  
30                     byte sendData[];  
31                     System.out.print("Server: ");  
32                     stgui = in.readLine();  
33                     sendData = stgui.getBytes();  
34                     DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, ipClient, portClient);  
35                     socket.send(sendPacket);  
36                 }  
37                 if (stgui.equalsIgnoreCase("bye") || stnhan.equalsIgnoreCase("bye")) {  
38                     ketthuc = true;  
39                 }  
40             } while (!ketthuc); //ketthuc=false socket.close(); }  
41         } catch (UnknownHostException e) {  
42             e.printStackTrace();  
43         }  
44     }  
45 }
```

Bài tập 1: **BAI07-BT01**

Viết chương trình minh họa giao thức FTP cho phép client tạo kết nối gửi file lên server sử dụng UDP socket.

Hướng dẫn:

- Các file được gửi có thể có dung lượng (bytes) khác nhau, đối với UDP, số bytes tối đa là **65508** (65kb), do vậy chúng ta cần chia nhỏ file thành nhiều phần để gửi đi.
- Trước khi gửi file từ client lên server, client phải thông báo cho server biết rằng đang chuẩn bị gửi file với các thông tin như tên file, file được chia ra thành bao nhiêu phần, mỗi phần có dung lượng là bao nhiêu... các thông tin này được đóng gói trong đối tượng FileInfo.
- Ví dụ: chia file ra thành các phần có dung lượng 32kb, và phần cuối cùng có dung lượng là phần dư của tổng dung lượng chia cho 32kb.

Bài tập 1: **BAI07-BT01**

Các lớp hiện thực:

FileInfo:

- String destinationDirectory
- String sourceDirectory
- String filename
- long fileSize
- int piecesOfFile
- int lastByteLength
- String status
- Các phương thức get, set dữ liệu

Server:

- Mở serverSocket
- Lắng nghe kết nối từ client
- Nhận file truyền lên từ client

Client:

- Tạo kết nối đến server
- Gửi file lên server

Bài tập 2: **BAI07-BT02**

Các chức năng tương tự Bài tập 1 và có thêm giao diện cho phép người dùng lựa chọn file cần gửi lên server.

Bài tập 3: **BAI07-BT03**

Các chức năng tương tự Bài tập 2 và có thêm giao diện cho phép người dùng lựa chọn file cần tải về từ server.

Client

